

Reviewer Pack — Minimal Root Separability and Communication Complexity Growth...

Entry 887e0a8e37b8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 21:07:32 UTC

Minimal Root Separability and Communication Complexity Growth

Entry ID: 887e0a8e37b8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 21:07:32 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Number Theory **Field B** (complexity object): Communication Complexity

Statement:

For any given Boolean function f with n variables, the communication complexity $C(f)$ is linearly correlated with the minimal root separability of its associated algebraic variety V_f , such that $C(f) = \Theta(\min_root_separability(V_f))$.

Rationale (proposer's reasoning):

Algebraic number theory provides a rich framework to study algebraic varieties and their properties. The minimal root separability of an algebraic variety has been used in studying the complexity of problems related to polynomial equations. Communication complexity measures the amount of communication required for two parties to compute a function. Our conjecture bridges these two areas by positing that the communication complexity is related to the geometric property of the associated algebraic variety.

Taxonomy category: communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5a048bff8e0e5a29

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across 30 independent trials, the correlation coefficient between the minimal root separability and communication complexity of Boolean functions f is ≥ 0.7 . The criterion is falsified if the correlation coefficient is < 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - ('algebraic number theory' AND 'communication complexity') - ('minimal root separability' AND 'Boolean function communication complexity') - (' $C(f) = \Theta(\min_root_separability(V_f))$ ' AND 'algebraic variety')

Top relevant hits considered: - [http://arxiv.org/abs/0709.0640v1] Algebraic theta functions and Eisenstein-Kronecker numbers - [http://arxiv.org/abs/math/0404268v2] Simultaneous approximation by conjugate algebraic numbers in fields of transcendence degree one - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/1605.01142v2] Separations in communication complexity using cheat sheets and information complexity - [http://arxiv.org/abs/quant-ph/0607174v1] Exponential Separation of Quantum and Classical One-Way Communication Complexity for a Boolean Function - [http://arxiv.org/abs/1512.01210v1] Nearly optimal separations between communication (or query) complexity and partitions - [http://arxiv.org/abs/2603.25938v1] Narrowband searches for continuous gravitational waves from known pulsars in the first two parts of the fourth LIGO-Virgo - [http://arxiv.org/abs/2512.17990v2] Constraints on gravitational waves from the 2024 Vela pulsar glitch

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.5s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity(f):
        n = len(f)
        # Simplified model of communication complexity
        return n

    def minimal_root_separability(V_f):
        # Placeholder function to simulate minimal root separability calculation
        # This is a dummy implementation and should be replaced with actual computation
        return random.random()

    n_max = 40
    instances_tested = 30
```

```

metric_values = []
conjecture_holds = True

for _ in range(instances_tested):
    f = generate_boolean_function(random.randint(5, n_max))
    C_f = communication_complexity(f)
    r_s = minimal_root_separability(f)

    if math.isnan(r_s) or math.isinf(r_s):
        conjecture_holds = False
        counterexample = "minimal_root_separability_result_invalid"
        break

    metric_values.append(C_f / r_s)

if not conjecture_holds:
    return {
        "metric_name": "communication_complexity_over_minimal_root_separability",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

mean_C = sum(metric_values) / len(metric_values)
std_C = math.sqrt(sum((x - mean_C)**2 for x in metric_values) / len(metric_values))

return {
    "metric_name": "communication_complexity_over_minimal_root_separability",
    "metric_value": mean_C,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": mean_C >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(seed) for seed in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = random.sample(primes, 30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_C = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_C = math.sqrt(sum((r["metric_value"] - mean_C)**2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_C} std={std_C} support_fraction={support_fraction}")

```

```

elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"support_fraction_too_low\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction} < 0.8")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the correlation coefficient could not be calculated to determine if the conjecture is supported or falsified. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	31880
2	propose	ollama_remote	glm4:latest	0	0	16257
3	propose	ollama_remote	glm4:latest	0	0	13405
4	preregistration	ollama_remote	glm4:latest	0	0	13540
5	novelty	ollama_remote	glm4:latest	0	0	12224
6	novelty	ollama_remote	glm4:latest	0	0	12222
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14042
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34964
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7117
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38165
11	judge	ollama_remote	glm4:latest	0	0	31059

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 224876 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in [research/audit/887e0a8e37b8.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/887e0a8e37b8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/887e0a8e37b8.tar.gz` (if generated)